

HITCRT 视觉组代码编写规范

编写：黎林

修订：孙静文 张志刚

补订：方思安

审核：刘天

2022 年 3 月 13 日

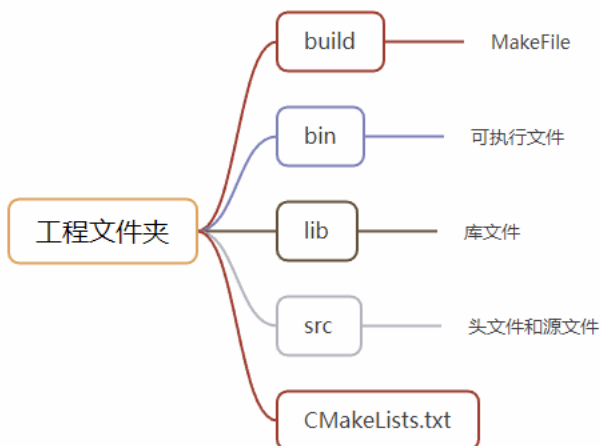
赠言

Always code as if the guy who ends up maintaining your code will be a violent psychopath who knows where you live.

—John F. Woods, 1991, in a post to the comp.lang.c++ newsgroup

一、工程规范

1. [推荐] 推荐工程目录结构如下：可按需添加其它文件夹。



（注：src 文件夹内不同模块放在不同的文件夹内，便于添加子模块）

2. [强制] 所有文件采用 UTF-8 编码。

二、风格规范

1. [强制] 使用 clang-format 格式化代码，风格统一为 Google，缩进宽度设为 4。
2. [强制] 除 main.cpp 外其它源文件都有对应的同名头文件，用于申明接口，函数声明必须放在头文件中，实现必须放在源文件中。
3. [强制] 缩进采用四个空格，若要使用 Tab 请先在编辑器里面将 Tab 设置为四个空格。
4. [强制] 代码中禁止使用 using namespace cv 之类的全局命名空间，而是使用域解析符例如 cv::Mat；这样的好处是可以避免名称冲突以及代码可读性强。
5. [强制] 函数签名中形参顺序：输入在前，输出在后。
6. [强制] 类的申明顺序依次为：public, protected, private。
7. [强制] 不会被修改值的函数形参或变量，加 const。

8. **[强制]** 仅初始化一次的或固定值的成员变量，声明时加 `const`。注：此时变量只能通过构造函数赋值，无法通过其他函数修改其值。
9. **[强制]** 子类被重写的函数，前加 `virtual`，后加 `final` 或 `override`，减少犯错同时增加易读性。
10. **[强制]** 除 `for` 等特殊语句，一行内不得有多条语句。
11. **[强制]** 预处理命令不得缩进，即它使位于缩进的代码块中。
12. **[强制]** 对 `for`，`while`，`if`，`if...else...`，`if...else...if...` 等语句始终使用花括号，即使只有一条语句。
13. **[强制]** 对于 `if...else...`，`if...else...if...` 等语句如果语句中的一部分使用了花括号，那么其他部分必须使用花括号。例如：

```
if ()
{
    ...
}
else
    ...    //错误，前面使用了花括号 else 后面也应该使用花括号。
```

14. **[强制]** 空的循环体必须使用 `{}` 而不是一个分号。例如：
- ```
while(); //错误，空循环体不能用分号表示。
while(){}; //正确，用“{}”表示空的循环体。
```
15. **[强制]** 头文件必须加 `#pragma once` 进行保护，用于替代 `#ifndef`。注：若个别环境下编译器不支持该宏定义，则改回 `#ifndef` 方法进行保护，格式 `<PROJECT>_<PATH>_<FILE>_H`。
16. **[强制]** 未完成或有待修改或优化的代码用 `“//TODO”` 标记，可以在注释后面加上作者姓名联系方式以及期望的完成日期。
17. **[强制]** 函数小于 10 行甚至更少才可以定义为内联函数，复杂内联函数，放在后缀名为 `-inl.h` 的头文件中。
18. **[强制]** 函数原型，符号常量，结构声明，类声明，模板声明，内联函数应放在头文件中，除内联函数外不得在头文件放置实现。
19. **[强制]** 参数表优先使用 OpenCV 自带的 XML/YAML 读取。
20. **[推荐]** 不被修改值的函数形参输入的数据类型，如 `int`、`double`、`float` 这种，只加 `const` 即可。如果是 `string` 或者别的大于八字节的类型（比如 `Armor`，或者 `LightBar`），加 `const` 的同时加 `reference “&”` 减少拷贝。
21. **[推荐]** 向 `vector` 中增加元素之前，若已知元素个数，先 `reserve` 预留容量，减少 `table grow cost`。
22. **[推荐]** 尽量避免使用基本数据类型，如数组，以 `STL` 等库中封装过的数据类型代替。
23. **[推荐]** 尽量用智能指针 (`shared_ptr` 或 `unique_ptr`) 代替普通指针，免除内存泄漏等问题的困扰。
24. **[推荐]** 主函数只保留各模块入口或初始化命令。
25. **[推荐]** 代码一行长度不超过 80 字符。
26. **[推荐]** `#include` 的路径及顺序。
- `dir/foo.cpp` 或 `dir/foo_test.cpp` 的主要作用是实现或测试 `dir2/foo2.h` 的功能，`foo.cpp` 中包含头文件的次序如下：
- 1) `dir2/foo2.h`
  - 2) C 系统文件

- 3) C++ 系统文件
- 4) 其他库的 .h 文件
- 5) 本项目内 .h 文件

例如:

google-awesome-project/src/foo/internal/fooserver.cpp 的包含次序如下

```
#include "foo/public/fooserver.h" // 优先位置
```

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
#include <hash_map>
```

```
#include <vector>
```

```
#include "base/basictypes.h"
```

```
#include "base/commandlineflags.h"
```

```
#include "foo/public/bar.h"
```

27. [推荐] 使用前置声明尽量减少.h文件中#include的数量。

举例说明: 头文件中用到类 File, 但不需要访问 File 的声明, 则头文件中只需前置声明

class File; 无需#include "file/base/file.h"。

在以下情况做到使用类 Foo 而无需访问类的定义:

- 1) 将数据成员类型声明为 Foo \* 或 Foo &;
- 2) 参数、返回值类型为 Foo 的函数只是声明 (但不定义实现);
- 3) 静态数据成员的类型可以被声明为 Foo, 因为静态数据成员的定义在类定义之外。

例如:

```
#include <iostream>
```

```
using namespace std;
```

```
class B; // 这是前置声明 (Forward declaration)
```

```
class A
```

```
{
```

```
private:
```

```
 B* b;
```

```
public:
```

```
 A(B* b):b(b)
```

```
 {
```

```
 }
```

```

```

```
}
```

28. [推荐] 代码若需重复使用, 则应该将其定义为函数, 以减少代码冗余度并且使代码语义清晰。

29. [推荐] 函数不要过长, 应按照实现功能将其细分为多个较短函数便于阅读, 最好一个函数只做一件事。

30. [推荐] 建议按照功能将代码放到不同文件中, 比如将算法实现与调用算法功能的逻辑分开, 逻辑处要目的明确简明, 这样使代码可读性增强。

### 三、命名规范

1. **[强制]** 对任何对象（文件、变量、函数）命名时，除循环变量等外，使用有意义的名字；除附录中收录的或极常用的缩写外，命名中使用完整的单词。

例如： `int numErrors;` //正确，使用有意义的命名

`int n;` //错误，使用无意义的名字

2. **[强制]** C++文件以 `.cpp` 结尾，头文件以 `.h` 结尾。

3. **[强制]** 主文件命名为 `main.cc` 或 `main.cpp`。

4. **[强制]** 文件名与类名相同使用大驼峰，例如： `MyUsefulClass.cpp`；文件夹名用小写加下划线，例如： `aim_assist`。

5. **[强制]** 代码中使用 `hitcrt` 作为命名空间名称。

6. **[强制]** 宏常量字母全部大写，单词间以“`_`”隔开。

7. **[强制]** 类，结构体，枚举名命名为大驼峰型，枚举值全大写并用“`_`”隔开，例如：

```
class UrlTable { ...};
enum UrlTableErrors
{
 OK = 0,
 ERROR_OUT_OF_MEMORY,
 ERROR_MALFORMED_INPUT,
};
```

8. **[强制]** 普通函数和成员函数名均采用驼峰命名法，成员函数无需加前缀，例如：

```
getArmors();
setImageWidth();
```

9. **[强制]** 除特殊情况外，变量名一律采用驼峰命名法，类的成员变量以“`m_`”开头，静态变量以“`s_`”开头。

例：

```
smallArmor //局部变量
m_smallArmor //成员变量
s_smallArmor //静态变量
```

10. **[强制]** 特例 1：全局变量和常量，变量名全大写，用下划线分割。

例：

```
int IMAGE_WIDTH;
const double GRAVITY_ACCELERATION = 9.8;
```

11. **[强制]** 特例 2：从参数表中读入加载的静态变量，实际功能与全局变量一致的，也按照全大写、下划线分割形式命名。

例：

```
Param::GRAVITY_ACCELERATION
```

12. **[强制]** 特例 3：若使用智能指针或封装过的类名中含有 `Ptr` 的类，变量名末尾加 `Ptr`；普通指针不加前缀/后缀。

例：

```
std::shared_ptr<hitcrt::Armor> armorPtr; //智能指针
std::shared_ptr<hitcrt::Armor> m_armorPtr; //智能指针成员变量
ICameraPtr cameraPtr; //含 Ptr 后缀的类名
char * fileName; //普通指针
```

13. [推荐]尽量少使用全局变量和外部变量，以静态变量代替。
14. [推荐]类型和变量名一般为名词：如 `fileOpener`、`numErrors`；函数名通常是指令性的，如 `openFile()`、`setNumErrors()`。
15. [推荐]名称中 `for` 可用 4 代替，`to` 可用 2 代替，例如 `timeForRun` 简写成 `time4Run`，`intToChar` 可写成 `int2char`。

## 四、代码注释

1. [强制]使用 Doxygen 格式书写源文件注释，可用 `vscode` 或 `clion` 自动生成模板。
2. [强制]比赛结束后所有源文件开始处必须加上注释，格式如下：

```
/**
 * @file 文件名
 * @brief 简述功能
 * @author 作者(联系方式)
 * @date 创建日期
 *
 * @copyright Copyright (C) 20XX, HITCRT_VISION, all rights
reserved.
 *
 * @par 修改日志,可选
 * <table>
 * <tr><th>Date <th>Author <th>Description
 * <tr><td>2021-12-18 <td>BG2EDG <td>
 * </table>
 */
```

3. [强制]函数实现前必须有注释，备赛期间保证基本可读性即可，格式如下：

```
/**
 *@brief <简述>
 *@param <参数说明>
 *@return <返回值>
 *@author <作者姓名>
 *@mail <邮箱，谁写谁全责>
 */
```

4. [推荐]函数声明及全局变量声明前添加注释简要说明用途。
5. [推荐]在类定义前面加上注释说明类的功能和用法。

## 五、版本控制

1. [强制]git 备份周期：本地 git 每天都需要进行备份，码云 git 每星期一次，并在 README.md 中简要说明进度，指导会定期检查。本地 git 的 commit 次数不要太多，小修改不要 commit，否则 commit 太长不便于查找。
2. [强制]每次 commit 写清楚修改了什么，达到了什么样的结果。
3. [强制].gitignore 中将编译产物(build 文件夹和 bin 文件夹)、大文件(.jpg 和 .avi)和 IDE 生成的配置文件夹(.vscode 和 .clion)忽略，**禁止将大文件上传码云**，

会影响 pull。

附录

| 名称     | 缩写            | 全拼或相关信息       |
|--------|---------------|---------------|
| 原图像    | src_img       |               |
| 显示图像   | show_img      |               |
| 灰度图    | gray_img      |               |
| 二值图    | bin_img       |               |
| 步兵机器人  | infantry      |               |
| 英雄机器人  | hero          |               |
| 基地机器人  | base          |               |
| 哨兵机器人  | sentry        |               |
| 工程机器人  | engineer      |               |
| 空中机器人  | drone         |               |
| 补给机器人  | bulletstation |               |
| 裁判系统   | referee       |               |
| 装甲     | armor         |               |
| 装甲片灯条  | bar           |               |
| 俯仰     | picth         |               |
| 偏航     | yaw           |               |
| 小神符    | small_buff    |               |
| 大神符    | big_buff      |               |
| 九宫格    | nine_quads    |               |
| 九宫格格子的 | quad          |               |
| 数码管数字  | code_num      |               |
| 像素     | pix           | pixel         |
| 控制     | ctrl          | control       |
| 参数     | param         | parameter     |
| 缓冲     | buff          | buffer        |
| 时钟     | clk           | clock         |
| 指令     | cmd           | command       |
| 比较     | cmp           | compare       |
| 配置     | config        | configuration |
| 错误     | err           | error         |
| 增量     | inc           | increment     |
| 信息     | msg           | message       |
| 前一个    | prev          | previous      |

|       |      |            |
|-------|------|------------|
| 临时    | tmp  | temporary  |
| 系统    | sys  | system     |
| 平均    | avr  | average    |
| 计数    | cnt  | count      |
| 总和    | sum  | summary    |
| 差分/差异 | diff | difference |